

Professores:

Celso Rodrigo Giusti (The Best)

Daniel Manoel Filho

Felipe Augusto Stevanim Gregate

Marlon Palata Fanger Rodrigues

NÍVEIS DE TESTE



O que são

Testes Funcionais



Testes funcionais verificam o que o software faz, testando regras de negócio e entradas e saídas esperadas (como testar se o botão de login funciona).

O que são

Testes Não Funcionais



Testes não funcionais avaliam como o sistema se comporta, medindo qualidade como velocidade, segurança e facilidade de uso (como medir se a tela abre em menos de um segundo)

O que são

Níveis de Teste



Os níveis de teste representam as diferentes "camadas" em que o software é testado, do menor pedaço de código até o sistema completo em uso real.

- Cada nível testa uma parte diferente: da função isolada até a experiência do usuário final
- Quanto mais avança nos níveis, mais próximo do uso real o teste fica
- Ordem: Unitário → Integração → Sistema → Aceitação

Teste Unitário

Testa a menor unidade possível do código isoladamente — uma função, um método, uma classe.

- Feito pelo próprio desenvolvedor, geralmente durante a codificação
- Não depende de outras partes do sistema estarem prontas
- Pega erros de lógica cedo, antes de virar um problema maior

Exemplo:

Testar se a função *calcularFrete(peso, distancia)* retorna o valor correto para diferentes combinações de peso e distância, sem se importar com o resto do app.

Teste Unitário + IA

Ferramentas de IA vêm mudando como testes unitários são escritos no mercado.

- Copilot, ChatGPT e ferramentas similares conseguem sugerir ou gerar casos de teste a partir do código;
- Ajudam a lembrar cenários que o dev esqueceria (valores negativos, campos vazios, casos extremos);
- Não substituem o entendimento do dev sobre o que precisa ser testado — só aceleram a escrita.

Testes de Integração

Testa se módulos ou componentes diferentes, já unidos, funcionam corretamente juntos.

- Cada módulo pode passar no teste unitário e ainda assim falhar quando combinado com outro
- Foca na comunicação entre as partes: troca de dados, chamadas entre módulos, integrações com API/banco

Exemplo:

Testar se o módulo de "Pedido" consegue chamar corretamente o módulo de "Pagamento" e receber a confirmação de volta, mesmo que cada um isoladamente já tenha passado nos próprios testes unitários.

Exercício: Testes de Integração

Sistema: app de pedidos com módulos separados de Carrinho, Pagamento e Estoque.

Tarefa: dê 1 exemplo de teste unitário (testando 1 módulo isolado) e 1 exemplo de teste de integração (testando 2 módulos se comunicando) para esse sistema.

Testes de Sistema

Testa o sistema completo e integrado, do início ao fim, simulando o uso real — não mais partes isoladas.

- Verifica o comportamento do sistema como um todo, incluindo fluxos completos;
- Cobre tanto requisitos funcionais quanto não-funcionais (performance, segurança, usabilidade);
- Geralmente feito pela equipe de QA, num ambiente parecido com o de produção.

Exemplo:

Testar o fluxo completo de um app de pedidos: usuário faz login, navega pelo cardápio, adiciona itens ao carrinho, aplica cupom, escolhe forma de pagamento e finaliza o pedido — tudo em sequência, **como um usuário real faria**.

Testes de Aceitação

Verifica se o sistema atende aos requisitos e expectativas do cliente/usuário final – é o último nível antes da entrega.

- Responde: "isso é o que o cliente pediu e precisa?"
- Costuma ser feito pelo próprio cliente, por usuários reais, ou por QA representando o cliente
- Usa critérios de aceite definidos previamente (o que precisa estar certo para o sistema ser aprovado)

Exemplo: O cliente de um restaurante que encomendou o app de pedidos testa se consegue realmente fazer um pedido do jeito que imaginou

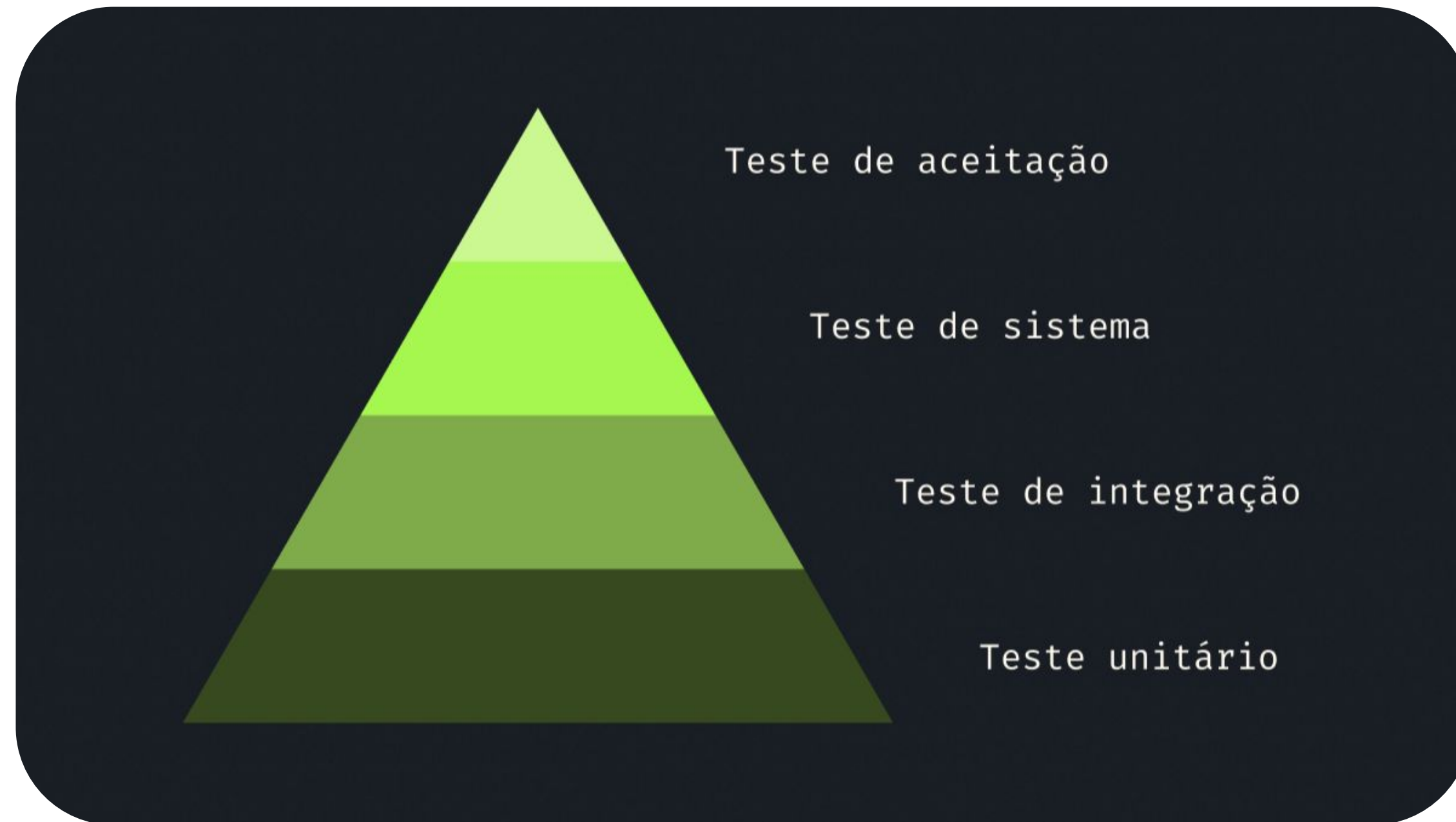
Recapitulando



- Unitário: 1 função/método isolado — feito pelo dev
- Integração: módulos se comunicando — feito pelo dev/QA
- Sistema: sistema completo, fluxo real — feito pelo QA
- Aceitação: atende ao que o cliente pediu? — feito pelo cliente/usuário final

Níveis de Teste

Pirâmide de Testes

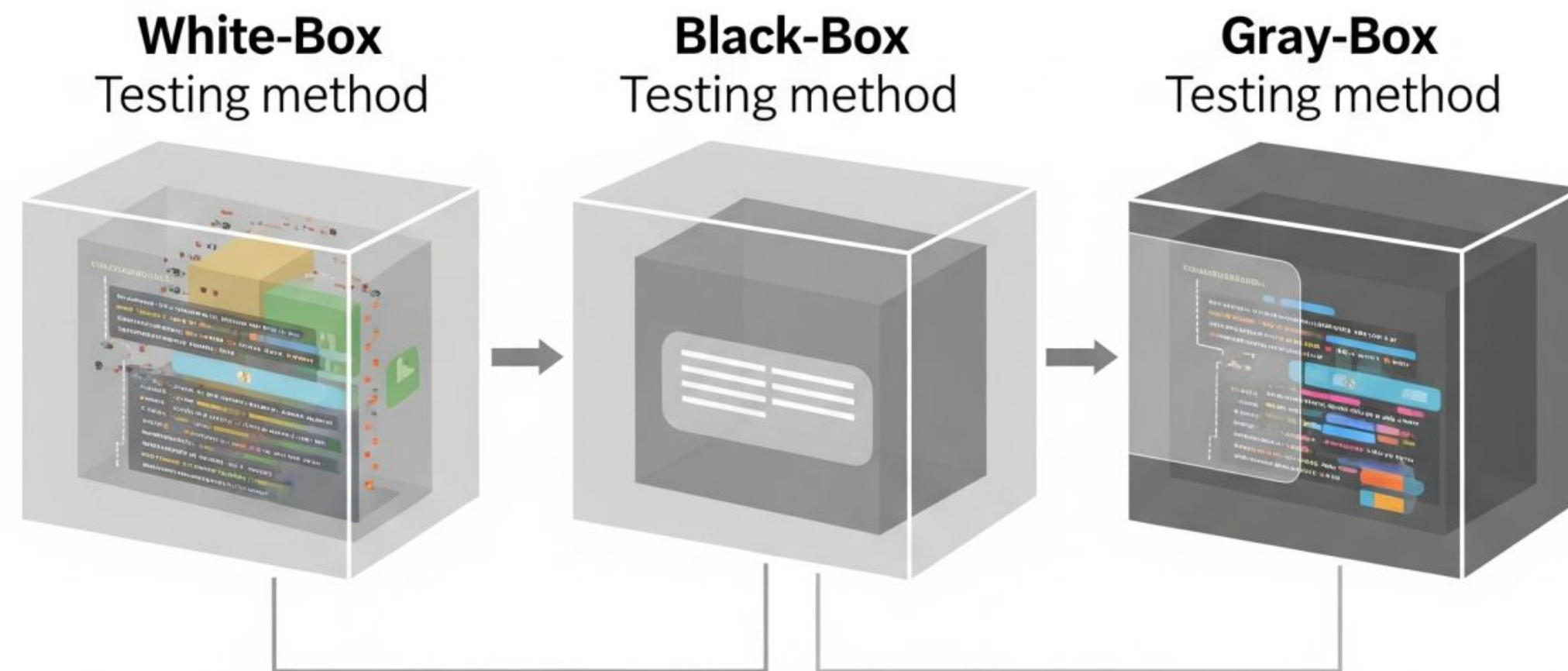


Conceito

Caixas de Teste



Além dos níveis, temos também as caixas de teste que é um conceito que determina quanto o testador sabe sobre o código na hora de testar.



Caixa Branca

O tester conhece e enxerga a estrutura interna do código e testa com base nela.

- Analisa a lógica interna: condições, loops, caminhos possíveis dentro do código;
- Geralmente feito por quem programou (ou tem acesso total ao código);
- Muito ligado ao teste unitário e de integração.

Caixa Preta

O testador não conhece o código por dentro — testa apenas entradas e saídas, do ponto de vista de quem usa o sistema.

- Não importa como o sistema calcula o resultado, importa se o resultado está certo;
- Simula a perspectiva do usuário final;
- Muito ligado ao **teste de sistema e de aceitação**.

Caixa Cinza

Uma combinação das duas: o testador tem conhecimento parcial da estrutura interna, mas testa principalmente pela perspectiva funcional/externa.

- Usa o conhecimento interno para decidir melhor o que testar, mas não valida linha a linha do código;
- Comum em testes de integração entre sistemas (ex: sabe que existe uma API por trás, mas testa via requisições, não via código-fonte).

Exercícios: Parte 1

Sistema: app de pedidos (Cantina Bella Vita).

Tarefa: para cada situação, diga se é Caixa Branca, Preta ou Cinza — e justifique em 1 frase o porquê, não só a classificação:

- a) O dev testa se todos os if/else de uma função de cálculo de troco são percorridos;
- b) O QA testa se o pedido é confirmado corretamente sem olhar o código, só usando o app;
- c) O QA sabe que existe uma validação de estoque no banco e testa cenários de concorrência via requisições, sem ver o código;

Exercícios: Parte 1

Sistema: app de pedidos (Cantina Bella Vita).

Tarefa: para cada situação, diga se é Caixa Branca, Preta ou Cinza — e justifique em 1 frase o porquê, não só a classificação:

d) Um dev revisa o código de um colega e, ao ver que existe um try/catch silencioso, decide testar propositalmente um cenário de erro de rede, mesmo usando só a interface do app pra isso.

Exercícios: Parte 2

Sistema: app de pedidos vai lançar uma funcionalidade nova: **agendamento de pedido para horário futuro.**

Para essa funcionalidade nova, definam:

- Um teste de nível Unitário (o que testar isoladamente?);
- Um teste de nível de Integração (quais módulos precisam se comunicar aqui?);
- Um teste de nível de Sistema (qual o fluxo completo, ponta a ponta?);
- Um critério de aceitação que o cliente do restaurante definiria para aprovar essa funcionalidade.

Para cada um dos 4 itens, digam também se o teste seria mais naturalmente Caixa Branca, Preta ou Cinza — e por que.

Exercícios: Parte 3

Situação: Um colega de equipe argumenta o seguinte:

"Não faz sentido gastar tempo com teste de nível de Sistema se todos os módulos já passaram nos testes de Integração, se as partes se comunicam bem, o sistema todo funcionando é consequência."

Tarefa: vocês concordam com esse argumento? Deem um exemplo de bug que passaria por todos os testes de integração (módulos combinados dois a dois) mas ainda assim quebraria no teste de Sistema (fluxo completo com 3+ módulos envolvidos ao mesmo tempo).